駕馭 Scala 型別系統：在型別層打造自動機

Exploiting Scala Type System: Building Type-Level Automaton

Speaker: Kyle Lin (ChAoS)

Table Of Contents

- Introduction to Type Level Programming
- Building Rule 110 Automaton
- Applications
- Conclusion

Introduction to Type Level Programming

General Type Level Programming

- Compile-Time Evaluation:
  Evaluate once,
  no overhead in runtime!
- Strict Logic Check:
  Let compiler check for you,
  no ambiguity!



Scala Type Level Programming

- Scala 3 introduces few type operations allow us to type level programming more conveniently!
  - **Type matching**
  - Macro
  - Union and Intersection Types
  - **Contextual Abstractions**
  - And more...!

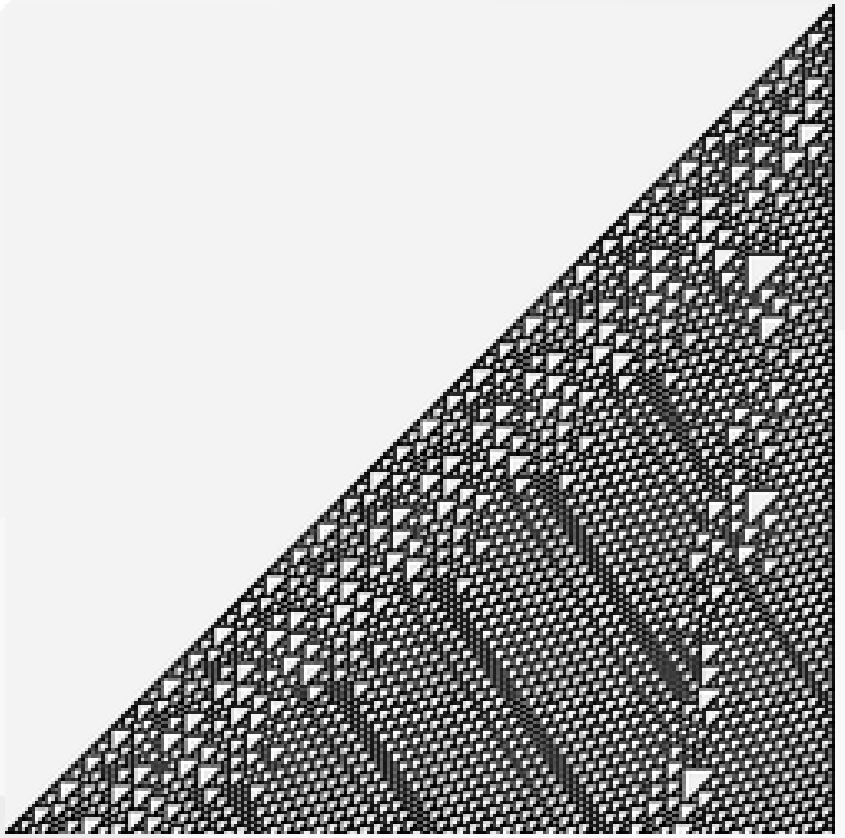


Building Rule 110 Automaton

Rule 110

- Known as Rule 110 cellular automaton
- Elementary cellular automaton
- Turing Complete!

111	110	101	100	011	010	001	000
0	1	1	0	1	1	1	0



Structure

Our type-level Rule 110 program consists of following minimum definition:

- Natural Number ($\mathbb{N}$)
- Cell (binary state, e.g. 1 or 0)
- heterogeneous List
- Rule 110 State Transformer
- Automaton (Itself)

Natural Number ($\mathbb{N}$)

Summarized from Peano Axioms:

- Starts from 0
- There's always an successor for a natural number

```
sealed trait Nat
class Zero extends Nat
class Suc[N <: Nat] extends Nat

// 0: Zero
// 1: Suc[Zero]
// 2: Suc[Suc[Zero]]
// and so on...
```

Cell

```
class I
class O
type Cell = I | O
```

Note: avoid using object to declare, as it will only declare it under term level, requires later type usages to need to add `.type` *reference*

Heterogeneous List

- Every types of elements in list may differ
- *Dependent Type*: Tracks the length of list in each node of list

```
sealed trait HList {
  // Length of list
  type N <: Nat
}
```

Heterogeneous List

```
case class HNil() extends HList {
  type N = Zero
}

case class ::[H, T <: HList]()(using val list: T) extends HList {
  type N = Suc[list.N]
}
```

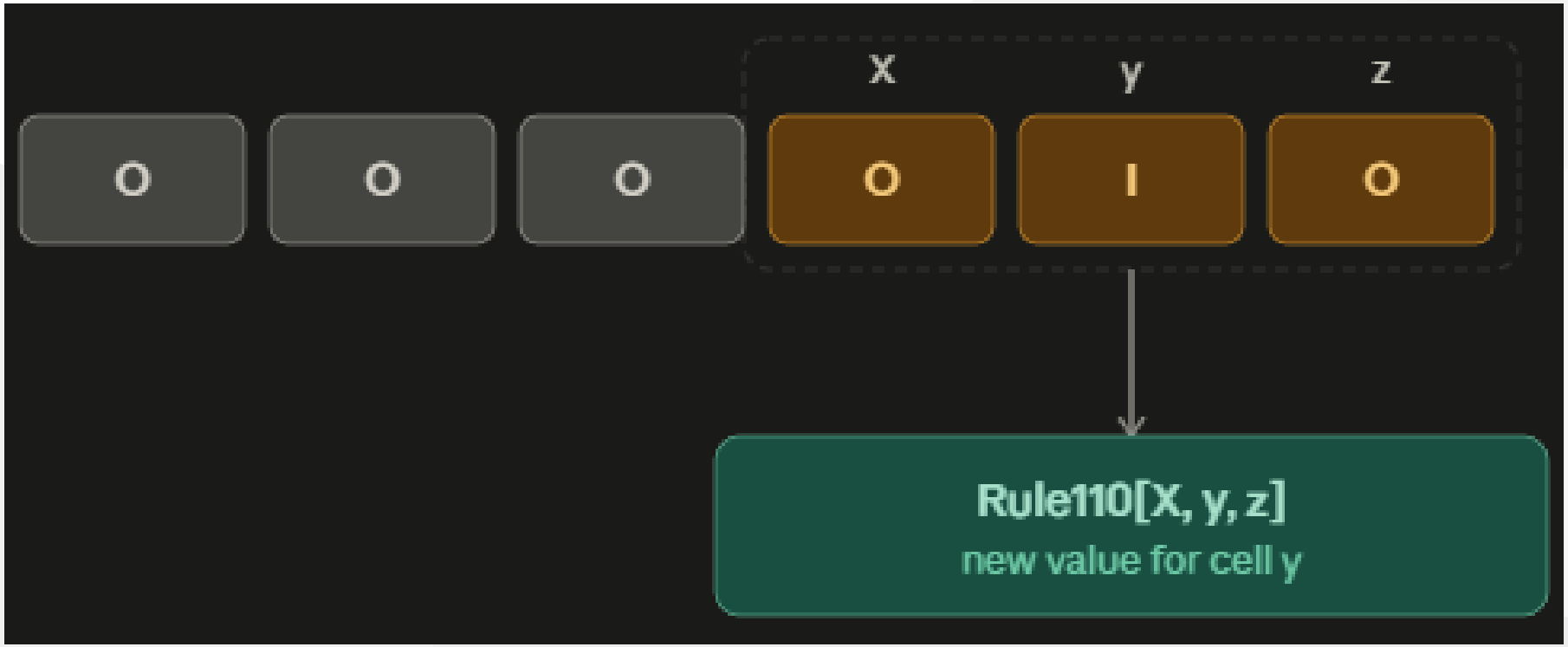
Bonus: This design can be exploited with invalid type being in the list, can you design a well-typed sized vector?

Rule 110 State Transformer

```
type Rule110[X <: Cell, Y <: Cell, Z <: Cell] = (X, Y, Z) match
  case (O, O, O) => O
  case (O, O, I) => I
  case (O, I, O) => I
  case (O, I, I) => I
  case (I, O, O) => O
  case (I, O, I) => I
  case (I, I, O) => I
  case (I, I, I) => O
```

Automaton

- Idea: Scan through the list with window size of 3



Automaton

- Generation Scanner / Generator

```
type Next[X <: Cell, Items <: HList] <: HList = Items match
  case y :: z :: rest => Rule110[X, y, z] :: Next[y, z :: rest]
  case y :: HNil => y :: HNil
  case HNil => HNil

type Start[Items <: HList] <: HList = Items match
  case x :: rest => x :: Next[x, rest]
  case HNil => HNil
```

Automaton

- Generation Iterator

```
type Gen[N <: Nat, State <: HList] <: HList = N match
  case Suc[n] => State :: Gen[n, Start[State]]
  case Zero   => HNil
```

Result

- A state-preserved Rule 110 Automaton Iteration Generator!

```
Gen[_2, O :: O :: O :: O :: I :: O :: HNil] =:=
  (O :: O :: O :: O :: I :: O :: HNil) ::
  (O :: O :: O :: I :: I :: O :: HNil) ::
  HNil

// _2 is shorthand type alias for Suc[Suc[Zero]]
```

Applications

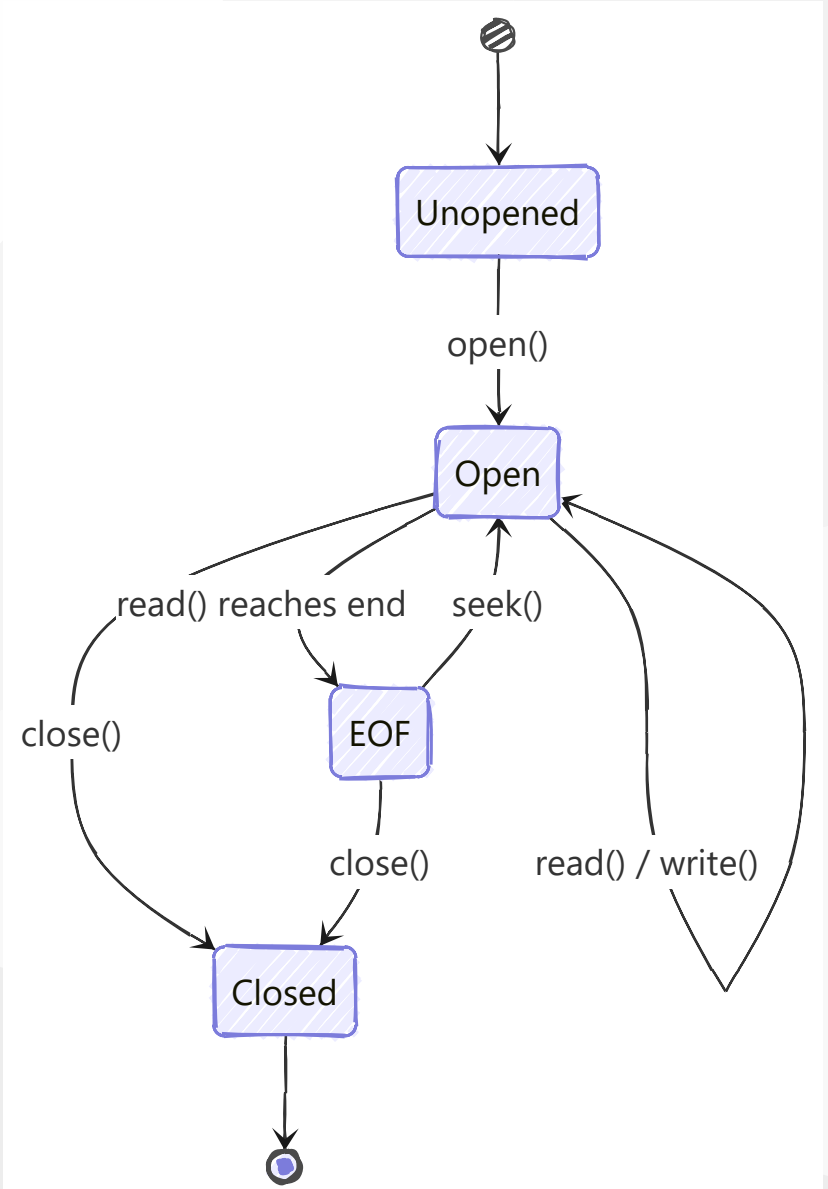
Reflection

So far we've covered how to program an automaton, Rule 110 in this case, directly in type level, but this is not meant to be in any production code...

Useful Applications, Pt.1: Stateful Handling

By embedding type information into function, and adding constraints, we can use type to construct a type-safe file stream class.

Worth to notice this contains usage of Phantom Type.



Useful Applications, Pt.1: Stateful Handling

- Rough sketch of type-safe filestate class implementation

```
sealed trait StreamState
sealed trait Closeable extends StreamState

class Unopened extends StreamState
class Open extends Closeable
class EOF extends Closeable
class Closed extends StreamState
```

Useful Applications, Pt.1: Stateful Handling

- Rough sketch of type-safe filestate class implementation

```
class FileStream[S <: StreamState] private (path: Option[String]):
  def open(p: String)(using S =:= Unopened): FileStream[Open] = ???

  def read()(using S =:= Open): (String, FileStream[Open | EOF]) = ???

  def write(data: String)(using S =:= Open): FileStream[Open] = ???

  def seek(pos: Int)(using S =:= EOF): FileStream[Open] = ???

  def close()(using S <:< Closeable): FileStream[Closed] = ???
```

Useful Applications, Pt.2: Simple Arithmetic

By embedding a simple type-level natural number ($\mathbb{N}$) along with bit vector, we can constrain how wide the bit vectors should be for each common bitwise (or arithmetic) operations.

- Example of binary multiplication (Long multiplication)

$$
\begin{array}{cccccc}
 & & & & 1 & 1 \\
\times & & & & 1 & 1 \\
\hline
 & & 1 & 0 & 0 & 1
\end{array}
$$

Useful Applications, Pt.2: Simple Arithmetic

- Rough sketch of type-safe bit vector implementation

```
sealed trait Nat
class Zero extends Nat
class Suc[N <: Nat] extends Nat

type Add[A <: Nat, B <: Nat] <: Nat = A match
  case Zero    => B
  case Suc[n]  => Suc[Add[n, B]]

class BitVector[N <: Nat](val v: Array[Boolean])

def multiply[N <: Nat](a: BitVector[N], b: BitVector[N]): BitVector[Add[N, N]] = ???
```

Pratical Considerations

When theory does not grant your wish

Theoretically, type level programming sounds nice for several reasons:

- Compile-time type correctness guarantee
- Zero cost abstractions in runtime
- Compositionality based on functional-implementation

When theory does not grant your wish

However, the actual implementation may not agree with the outcome:

- Confusing debug information on type error
- Significantly longer compilation time (based on compiler implementation)
- LSP may suffer from performance crisis
- **Not collaboration-friendly for non-functional writers**
- And possibly more...

Technical Balancing

For example, under the hood, Chisel does not use Type Level Programming generally.



Conclusion

Key Takeaway

- Type level programming encourages programmers to review implementation structurally before going for runtime debugging
- Construction effort is high, but rewards strong foundation towards function / class definitions
- Avoid large (or deeply-nested) type level programming as compiler may not be able to handle it

Q & A

- Rule 110 in Scastie Playground



- Slide

